

ownership_precondition

scripts/ownership_precondition.sh — Ownership Startup Precondition

Revision: 5 **Last modified:** 2026-08-26T15:15:25Z **Purpose:** Operator guide for the startup check that refuses to start the system when a declared location cannot produce operator-owned files. **Last verified:** 2026-08-26

Overview

scripts/ownership_precondition.sh answers exactly one question, for every location listed in config/owned_paths.yaml:

If this system writes a file here, will that file belong to the person who started the system?

If the answer is “no” anywhere, the script exits non-zero so startup can be refused. This is **fail-closed by operator decision** (clarify session, 2026-08-21): starting with a warning was explicitly rejected, because a missed warning silently reproduces the defect the feature exists to remove — content landing at an identity (uid 100999 under a rootless container mapping) that has no host account, so the operator could not read, move, or delete their own downloads, and could not read their own credential database.

Why it PROBES instead of inspecting

Ownership of a location cannot be read off the location itself. This is the single most important thing to understand about this script, and it is measured fact rather than caution: the download root was owned by the operator’s uid **and still received files at uid 100999**. That is the defect. So the directory’s own owner, the configured PUID, and “no error occurred” are all *proxies* for the condition — and §11.4.201 forbids asserting a proxy in place of the condition it stands for. A probe that passes because it never wrote anything is a false pass.

Every verdict this script emits therefore comes from creating a **real file** in the **real location** and reading the owner back from the host, then removing the file.

Why there are TWO probes, and why neither speaks for the other

Probe	What it does	What it proves
P2 (host write)	The script, running as the operator, creates a file in the location, reads its owner, removes it	The location exists, is writable, and is on a filesystem that carries ownership at all. Necessary, never sufficient.
P1 (container write)	A throwaway container writes into the location as the <i>service’s declared identity</i> , then the owner is read back from the host	The real condition. This is the only probe that can observe the reported defect.

P2 alone is not enough for a reason that was measured during Phase 1 of this feature: a host-side write by the operator produces an operator-owned file **even in the directory whose container writes land at 100999** — `probe_location` returned ok for exactly that directory. So the report keeps the two probes separate and never dresses a P2 pass up as a P1 pass.

Declared **files** (the credential store, `config/boba.db`) are the one carve-out: there is nothing to create beside them, and reading the target's own owner IS the real condition there rather than a proxy for it. Files get no P1 probe.

Prerequisites

- `bash 4+`, `stat`, `mktemp` (coreutils)
- `python3` with **PyYAML** — used to parse `config/owned_paths.yaml` and `docker-compose.yml`. The interpreter is *probed*, not assumed: a `python3` that cannot import `yaml` is treated as unusable, and the script exits 2 rather than producing a confusing downstream failure.
- `scripts/lib/ownership.sh` — sourced, never re-implemented (scope resolution, operator uid, and the probe itself live there and are shared with `scripts/ownership_repair.sh` and the pre-build gate).
- `config/owned_paths.yaml` — the declared scope (data-model **E1**).
- **Optional:** `podman` or `docker`. Without a container runtime the P1 probe cannot run; the script reports an honest SKIP rather than refusing, see *Edge cases*.

Usage examples

Manual invocations need `.env` loaded first — `start.sh` does this for you, a bare shell does not. `QBITTORRENT_DATA_DIR` is read from `.env` by `start.sh`'s `load_environment()` (§ *Wiring* below) before this script ever runs. Invoke this script directly from a shell that has not sourced `.env` and the `${QBITTORRENT_DATA_DIR:-/mnt/DATA}` placeholder in `config/owned_paths.yaml` falls through to its documented default — `/mnt/DATA` — which does not exist on every host. Measured on this host 2026-08-21 (see `specs/002-user-owned-downloads/quickstart.md` Scenario 4 for the same evidence run against the full quickstart):

```
$ env -u QBITTORRENT_DATA_DIR scripts/ownership_precondition.sh;
    echo "exit=$?"
...
- /mnt/DATA: declared (kind=downloads)
  but ABSENT, and it is not marked optional – E1 makes an
  absent non-optional path an error, not a skip
Startup refused. Fix the location(s) above, or run scripts/
ownership_repair.sh.
exit=1
```

That refusal is **correct** — `/mnt/DATA` genuinely does not exist on this host — but it is not the answer to “is the system healthy”, because the placeholder never resolved to this host’s real download root. Load `.env` first, exactly as `start.sh` does, and the same command against the same tree passes:

```
$ set -a; source .env; set +a
$ scripts/ownership_precondition.sh; echo "exit=$?"
...
OWNERSHIP-PRECONDITION: OK
exit=0
```

This applies equally to a manual `scripts/ownership_repair.sh` invocation (FR-004c): on a host where `/mnt/DATA` happens to exist but the real download root is elsewhere, a bare repair walks the **wrong tree** silently — it neither errors nor warns, because `/mnt/DATA` is a perfectly valid path, just not the operator’s actual download root. Load `.env` before any manual invocation of either script.

Example 1 — the normal invocation

```
scripts/ownership_precondition.sh
```

Prints a per-location verdict, then one of `OWNERSHIP-PRECONDITION: OK`, `... FAIL`, or `... CANNOT-RUN`. Run this way (or via `start.sh`, which loads `.env` first) — see the `.env` caveat above before running it from a bare shell.

Example 2 — quiet mode, for a startup path

```
scripts/ownership_precondition.sh --quiet
```

Suppresses passing detail and the success banner. It does **not** suppress a refusal, a cannot-run report, or a probe-coverage gap — see *Flags*.

Example 3 — check a different scope file

```
scripts/ownership_precondition.sh --scope /path/to/
    other_owned_paths.yaml
scripts/ownership_precondition.sh --scope=/path/to/
    other_owned_paths.yaml # equivalent
```

Both forms are accepted. `--scope` takes precedence over the `OWNED_PATHS_FILE` environment variable.

Example 4 — declare that there is no container runtime

```
CONTAINER_RUNTIME= scripts/ownership_precondition.sh
```

An explicitly **empty** CONTAINER_RUNTIME means “there is no runtime” and is honoured as such — P1 is skipped and no container is launched. This is how test suites and gates run the script without spawning containers. Contrast with the variable being **unset**, which means “not decided yet” and makes the script detect podman/docker itself.

Example 5 — use in a shell conditional

```
if scripts/ownership_precondition.sh --quiet; then
    echo "safe to start"
else
    rc=$?
    echo "refused (exit ${rc}) – see the report above"
fi
```

Note that 1 and 2 mean different things and must not be collapsed; see *Exit codes*.

Flags

Flag	Effect
--scope <path> / --scope=<path>	Scope file to check. Overrides OWNED_PATHS_FILE. A missing argument to the space-separated form is exit 2, not a silent default.
--quiet	Suppress per-location OK detail and the OWNERSHIP-PRECONDITION: OK banner. Refusals, cannot-run reports, and probe-coverage gaps are always printed.
-h, --help	Print usage and exit 0.
(anything else)	Unrecognised: usage to stderr, then exit 2. An unknown flag means the caller asked for something this script does not do; ignoring it would silently run a

Flag	Effect
	<i>different</i> check than the one requested.

Why `--quiet` hides only good news. A quiet flag that could hide a refusal — or hide the fact that the probe which observes the real defect never ran — would re-create the missed-warning failure the fail-closed decision was made to avoid. The implementation splits output into `say()` (silenceable) and `say_always()` (never silenced), and every failure path uses `say_always`.

Env vars

Variable	Meaning
OWNED_PATHS_FILE	Scope file path when <code>--scope</code> is absent. Read by <code>scripts/lib/ownership.sh</code> ; default <code>config/owned_paths.yaml</code> under the project root.
CONTAINER_RUNTIME	Set-but-empty = no runtime available (P1 skipped, nothing detected). Unset = detect podman then docker. Non-empty = use that command verbatim. Absent and empty are deliberately different states — this mirrors <code>start.sh</code> , which sets the empty value precisely to mean “none found”.
PYTHON_BIN	Consulted first by <code>ownership_python()</code> when choosing an interpreter (then <code>.venv/bin/python</code> , then <code>python3</code>). Each candidate must actually <code>import yaml</code> to be selected.
<i>scope placeholders</i>	Any <code>\${VAR:-default}</code> inside a declared path is expanded against the live environment — <code>QBITTORRENT_DATA_DIR</code> is the one that matters in the shipped scope, so the host-specific

Variable	Meaning
	download root resolves at run time instead of being hardcoded (§11.4.35). A manual invocation that has not loaded .env (set -a; source .env; set +a) sees the unset variable fall through to the default (/mnt/DATA), not this host's real download root — see the .env caveat at the top of <i>Usage examples</i> .

No credential value is ever read, printed, or logged by this script.

Exit codes

Code	Meaning
0	Every declared location can produce operator-owned files.
1	At least one cannot — startup MUST NOT proceed . Each offending location is named individually.
2	The check could not run at all.

2 is not a pass, and it is not a failure of the system either. It means the check asserted *nothing*: the scope file was missing, unparseable, or empty; **a declared entry resolved to no usable location** (a `${VAR}` with no `-` default that expanded to nothing, a missing path: key, a path that silently expanded into a *different* one, or a spelling outside the documented `${VAR} / ${VAR:-default}` grammar — a nested `${A:-${B}}`, `${}`, `${1}`, an unterminated `${VAR` — which would otherwise leave an unresolved marker in the path; see the same section in [ownership_repair.md](#)); **the scope declared a path the fence refuses** (see *The declared-path fence* below); no python3 with PyYAML was available; or the caller passed an unusable argument. When the scope reader is the one that refused, **its own message is replayed verbatim** in the CANNOT-RUN report: it is the only layer that still knows *which* entry failed and *why*, and a cause line that named only the file-level causes would state a cause that was never established. Reporting “could not run” as success is the §11.4.201(6) blind-instrument failure — a blind

instrument and a clean system return the same quiet zero — so this script refuses to conflate them. The CANNOT-RUN report always names the cause and ends with the literal line:

This check asserted NOTHING. It is not a pass – fix the cause above and re-run.

A caller that treats 2 as success has re-introduced the exact class of defect this feature exists to remove.

The declared-path fence

This script **creates a probe file inside every declared directory** — that is the whole point of it (see *Why it PROBES instead of inspecting*). Until BOB-187 nothing constrained what a declared path could **be**.
config/owned_paths.yaml ships

```
- path: "${QBITTORRENT_DATA_DIR:-/mnt/DATA}"
```

and .env — untracked by design, unreviewed, and in no review’s diff — supplies that variable. The same unreviewed value that drove a filesystem-wide find / in scripts/ownership_repair.sh (measured 2026-08-25, observed in ps and killed by pid) also selected where **this** script creates files. Lower blast radius than a recursive chown; the input is identical.

Before any probe runs, every declared entry is now passed through ownership_path_fence() in scripts/lib/ownership.sh — the **same** predicate the repair path uses. It is called, never reimplemented: §11.4.251 forbids a second dialect, and the reason the fence was put in the shared library when the repair was fenced is precisely so this caller would use that one. Its rules:

	Rule
F0	Lexical . / .. / // normalisation, with no symlink resolution — TOCTOU-free, and it must be able to judge a path that does not exist yet (.env, config/boba.db before first boot).
F1	A repo-relative entry must still resolve inside PROJECT_ROOT. An entry that climbs out with .. was

	Rule
	written to escape, and the scope format does not license that.
F2	An absolute entry needs at least 2 path components. The shipped default <code>/mnt/DATA</code> has exactly 2, so the configuration itself forces this floor.
F3	Never a system tree nor anything under one. <code>/usr/lib</code> has 2 components and clears F2, which is why F3 is separately load-bearing.
F4	Never the container runtime's own storage — the <i>computed</i> tree <code>\${XDG_DATA_HOME:-\$HOME/.local/share}/containers./home/<user></code> clears F2 and is deliberately not denylisted (<code>/home/<user>/Downloads</code> is an ordinary download root), so without F4 a declared root at or above the rootless image store is accepted by shape, and those files are owned by mapped subuids by design . A degenerate computed value — relative, or shallower than the F2 floor because <code>HOME</code> was unset — is discarded rather than used, so the rule can never broaden into a top-level prefix.

F4's honest limit: it resolves the default graphroot and an `XDG_DATA_HOME`-relocated one, because both are pure string operations. It does **not** read `storage.conf` or `CONTAINERS_STORAGE_CONF` — doing so would make the fence depend on filesystem state at check time, the very raceable property F0 exists to avoid. A host that has moved its graphroot is not covered by F4.

F0's honest limit: the fence judges the *spelling*; the kernel resolves the path. An existing symlink in a **non-final** component of a declared path is invisible to F0, and the walk (or the probe) lands under the link's target — measured, with no race required. A symlinked **final**

component is contained. What bounds the intermediate case is not the fence but who can write those components: they sit above the declared root, outside every container bind mount, and an actor able to write there can edit the untracked `.env` that supplies the root anyway. Tracked as BOB-159.

`/run`, `/home`, `/media`, `/mnt`, `/opt`, `/srv` and `/tmp` are **deliberately not** denylisted: this host's real library measures as `/run/media/<user>/<disk>/Downloads`, and a denylist that swallowed `/run` would refuse the live configuration — the §11.4.201(1) false-positive refusal the fence must not become.

One refused entry refuses the whole run, with exit 2, not 1. Exit 1 means “a declared location cannot produce operator-owned files” — a per-location verdict this check has *not* earned, because it deliberately never probed the path. Exit 2 already means “the check could not run” for a scope that is missing, unparseable or empty; a scope naming a path of this shape is the same class. Nothing is probed and no probe file is created anywhere:

```
$ CONTAINER_RUNTIME="" scripts/ownership_precondition.sh --scope /
tmp/root-scope.yaml
OWNERSHIP-PRECONDITION: CANNOT-RUN
- the declared scope names 1 path(s) this check must never
probe: /tmp/root-scope.yaml
- REFUSED / - '/' resolves to the filesystem root '/' - a
recursive ownership change of the whole filesystem is never a
declared scope
- nothing was probed, and no probe file was created anywhere
This check asserted NOTHING. It is not a pass - fix the cause
above and re-run.
```

Honest boundary (§11.4.6). The fence bounds the *shape* of a declared path. It cannot decide that a well-formed absolute path is the tree the operator *meant* — a deep absolute path outside the project is exactly what a download root is, so it must be accepted. What it removes is the class traced to a catastrophic outcome: filesystem roots, bare top-level directories, system trees, and `..` escapes.

Edge cases

- **A declared path that is absent and optional:** `true` → not a failure. Reported as absent, declared optional — not a failure and skipped. `config/boba.db` is exactly this shape before first boot, and this case is the contract's mandated **negative control**: a guard that

refuses a healthy system is as broken as one that passes a broken one (§11.4.201(1)).

- **A declared path that is absent and not optional** → failure (exit 1). Per data-model E1 an absent non-optional path is an error, not a skip.
- **No container runtime** → P1 is skipped with the named reason `runtime_unavailable`, and the script says out loud that nothing here asserts a service write would land at the operator's uid. It does **not** refuse: refusing every host without a runtime would be a §11.4.201(1) false-positive refusal, which is exactly as forbidden as a false pass.
- **P1 could not run for another reason** → also a named SKIP, never a refusal. The reasons are a closed set: `runtime_unavailable`, `route_unavailable` (compose file unreadable), `route_undeclared` (no compose service mounts this location), `service_image_unavailable` (no locally-present image, so reproducing the service would need a network pull at startup), `probe_container_failed(<service>: <first 120 chars of stderr>)`, and `probe_file_not_found(<service>)` (the container reported success but nothing landed on the host — the probe observed nothing, so it asserts nothing).
- **docker-compose.yml missing or unparseable** → routes are reported as route information unavailable, not as “no route declared”. An unread file and a file declaring nothing are different findings (§11.4.201(6)).
- **A compose service with no declared ownership route** → reported as NO route declared (neither `userns_mode: keep-id` nor `PUID=0`), but this alone does **not** refuse. A route reading is *configuration*, and the contract forbids inferring ownership from configuration; turning a config reading into a refusal would make this check refuse on a proxy, inside the check written to forbid proxies. Route **completeness** is enforced where it belongs: the pre-build gate `scripts/pre_build/check_cm_ownership_invariants.sh` (invariant 33, CM-OWNERSHIP-INVARIANTS).
- **An unrecognised verdict from either probe** is a failure, not a pass — “unrecognised is not clean”.
- **A probe file left behind**: both probes remove their probe file, including on the container-failure path. The host probe uses `mktemp` inside the probed directory; the container probe uses a `$$/RANDOM`-suffixed marker name.
- **Wiring, measured 2026-08-21T15:10Z**: `start.sh` calls this script from `run_ownership_gate()`, which runs on the normal start path **after** the directory-creation stages and **before** any container writes

(also on the `--recreate` path). The ordering is deliberate: the declared download root and `config/` are optional: `false`, so probing them before they exist would refuse a healthy fresh checkout — the §11.4.201(1) false-positive refusal. `start.sh` treats **both** exit 1 and exit 2 as refuse-to-start, and a missing script file as refuse-to-start; only exit 0 proceeds. It runs the script under `nice -n 19 ionice -c 3` when both tools are present. The systemd unit `scripts/systemd/user/boba-stack.service` deliberately does **not** declare its own `ExecStartPre` for this: it invokes `./start.sh --no-build` and inherits the single gate, so the two paths cannot contradict each other about whether the precondition ran. This wiring landed while this document was being written — it was measured directly in the working tree, not taken from the contract (§11.4.6).

Internal behaviour

1. **Parse arguments.** `--scope` is exported as `OWNED_PATHS_FILE` so the shared library sees it (the library's documented override, not a backdoor).
2. **Read the scope.** `ownership_scope_entries` emits one TAB-separated row per declared entry:
`<path>\t<kind>\t<optional>\t<preserve_mode>\t<recursive>.` A read failure is exit 2; a scope with zero locations is also exit 2 — “a precondition that checked zero locations has verified nothing”.
3. **Resolve the runtime** (see `CONTAINER_RUNTIME` above) and **read the compose routes** — one row per service carrying its image, `users_mode`, `PUID`, and normalised mount sources, with `${VAR:-default}` expanded *before* the `host:container` split (splitting first would tear a default value containing a colon in half).
4. **Per declared location:**
 1. Run **P2** via `probe_location`. Verdicts are `ok`, `absent`, `unwritable`, `wrong-owner:<uid>` (data-model **E4**).
 2. If the location is a regular **file**, stop here — its own owner is the real condition.
 3. Otherwise run **P1**: pick a compose service whose mount contains or is contained by this location *and* whose image is already present locally, then run `<runtime> run --rm --network=none [--users=<mode>] [--user <PUID>] --entrypoint /bin/sh -v <dir>:/ownership-probe:z <image> -c "touch ..." under a 120-second timeout, stat the resulting file on the host, and remove it. The image's own entrypoint is`

bypassed on purpose: the probe must perform a bare write, not boot the service.

5. **Print the probe-coverage report** whenever any P1 was skipped — always, `--quiet` included. It names each skipped location and reason, states that nothing there asserts a service write would land at the operator's uid, and prints the declared route as the clearly-labelled configuration-only fallback.
6. **Verdict.** Any failure → OWNERSHIP-PRECONDITION: FAIL, every offending location named, plus the actionable line `Startup refused`. Fix the location(s) above, or run `scripts/ownership_repair.sh`. Otherwise OWNERSHIP-PRECONDITION: OK.

Two implementation details worth knowing

- **Path containment is checked in both directions.** A service that mounts an *ancestor* of a declared location writes into it, and a service that mounts a *subdirectory* of it also writes into it. Matching one direction only would silently drop half the services that can produce files there.
- **TAB-separated rows are split with `readarray -d $'\t'`, not `IFS=$'\t' read`.** TAB is an IFS *whitespace* character, so `read` collapses a run of tabs into one delimiter — a row with empty middle fields (a compose service built from a Dockerfile has no `image`;) then shifts every later field left and the mounts column reads as `EMPTY`. Measured against this repository's own `docker-compose.yml` on 2026-08-21: the first version of the script used the collapsing `read` and reported “no compose service mounts this location” for `config/` and for the download root **while five services mount them** — a false statement produced by the instrument rather than by the system (§11.4.201(7)(c), “the path is part of the instrument”). The unit fixture scope has no compose service at all, so the suite could not see this; only running the real invocation against the real scope did.

Related scripts

- `ownership_repair.md` — `scripts/ownership_repair.sh`, the remediation this script's failure message points the operator at. The precondition **detects**; the repair **fixes**.
- `scripts/lib/ownership.sh` — the shared library both scripts source (`ownership_scope_entries`, `ownership_operator_uid`, `probe_location`, `ownership_scope_fingerprint`). Sourced rather than forked: a

second copy would be the near-identical fork §11.4.251 forbids and would drift from the repair and the gate that read the same scope.

- `scripts/pre_build/check_cm_ownership_invariants.sh` — invariant 33 (CM-OWNERSHIP-INVARIANTS) of `scripts/pre_build_verification.sh`. Enforces route **completeness** across compose services, which this script deliberately only reports.
- `config/owned_paths.yaml` — the declared scope (E1). Adding an entry here changes the scope fingerprint, which re-arms the repair.
- `tests/unit/test_ownership_precondition.sh` — the paired contract suite.
- `tests/ownership/test_container_writes_owned_files.py` — the §11.4.115 RED that captured the original defect at uid 100999.
- `../guides/file-ownership.md` — the operator-facing narrative for the whole ownership feature.

Cross-references

- `specs/002-user-owned-downloads/contracts/startup-precondition.md` — the contract this script implements (FR-010, FR-010a, FR-010b), including the 2026-08-21 amendment (finding X1) that split the single probe into P1/P2.
- `specs/002-user-owned-downloads/data-model.md` — **E1** scope, **E4** probe verdicts, **E5** service ownership routes.
- **§11.4.201** — every guard asserts the real condition; a false-positive refusal is a FAIL-bluff exactly as forbidden as a false-negative pass. This is the anchor behind the probe, behind the P1-skip-not-refuse rule, and behind exit 2 being distinct from exit 0.
- **§11.4.6** — no guessing: an unreadable scope is reported as unreadable, not as empty.
- **§11.4.3** — honest SKIP-with-reason, which is what an unavailable P1 emits.

Anti-bluff & §11.4 discipline

- Every verdict comes from a **file that was actually created and whose owner was actually read back**. No verdict is derived from a directory's own owner, from PUID, or from the absence of an error.
- The probe that can observe the real defect (P1) is never *implied* by the one that cannot (P2). When P1 does not run, the report says so, unconditionally, and labels the configuration fallback as configuration.

- 2 (cannot run) is structurally separate from 0 (pass), so a blind instrument can never be read as a clean system.
- --quiet cannot hide a refusal or a coverage gap.
- §11.4.263: this script signals no processes — there is no kill, pkill, or killpg anywhere in it, so the pgid <= 1 broadcast-kill hazard cannot arise.

Last verified

2026-08-25 (BOB-187) — the declared-path fence section was written against `ownership_path_fence()` as it actually exists in `scripts/lib/ownership.sh`, and the CANNOT-RUN transcript quoted there is pasted from a real run in this worktree, not composed. The four fence rules, the `MIN_COMPONENTS` value of 2, and the contents of `OWNERSHIP_FENCE_SYSTEM_TREES` (which does **not** contain `/run`, `/home`, `/media`, `/mnt`, `/opt`, `/srv` or `/tmp`) were read from that file rather than from its prose. The live scope was re-checked at the fence — all six shipped entries ACCEPT with the real `.env` value — without invoking the precondition against the real library.

2026-08-21 — every flag, exit code, environment variable, and skip reason in this document was read from `scripts/ownership_precondition.sh` and `scripts/lib/ownership.sh` at commit-time state, not from the script's own usage text alone. The `start.sh` wiring is described from the invocation actually present in the working tree at 2026-08-21T15:10Z (`run_ownership_gate()`, called at two sites), not from the contract's word for it.